🚀 План восстановления и продолжения работы iOS Eiros Agent

Дата создания: 22 сентября 2025
Статус: ✅ Готов к выполнению
Приоритет: Высокий

📊 Текущее состояние

✅ Что уже готово (95% проекта)

- **Полная архитектура iOS приложения** - все Swift файлы восстановлены
- **OpenRouter интеграция** - полностью функциональна с 12+ моделями
- **Touch Grid система** - готова к использованию
- **Python framework** - веб-автоматизация и Control Room
- **Документация** - полная и актуальная
- **Тесты** - базовая структура готова

🔄 Что требует доработки (5% проекта)

- **Реальная модель Qwen2-VL-2B** - сейчас используется mock
- **CoreML интеграция** - требует конвертации модели
- **Финальное тестирование** - проверка всех компонентов

🎯 План выполнения

Этап 1: Подготовка среды (10 минут)

```
# 1.1 Установка зависимостей для MLX
pip install mlx-vlm transformers torch pillow requests

# 1.2 Проверка доступности модели
python download_qwen2vl_mlx.py

# 1.3 Создание директории для моделей
mkdir -p EirosAgent/Resources/Models
```

Этап 2: Интеграция реальной модели Qwen2-VL (30-45 минут)

2.1 Загрузка и тестирование модели (15 минут)

```
# Загрузить MLX версию Qwen2-VL-2B
from mlx_vlm import load, generate

# Тестировать с образцом изображения
model, processor = load("mlx-community/Qwen2-VL-2B-4bit")
```

2.2 Создание Python-iOS моста (20 минут)

- Создать HTTP сервер для модели на Python

- Интегрировать с iOS через URLSession
- Заменить mock реализацию в `Qwen2VLManager.swift`

2.3 Обновление Swift кода (10 минут)

- Удалить mock функции
- Добавить реальные HTTP запросы к Python серверу
- Обновить обработку ответов

Этап 3: Оптимизация OpenRouter интеграции (15 минут)

3.1 Настройка API ключей

- Добавить инструкции по получению OpenRouter API ключа
- Протестировать все поддерживаемые модели
- Оптимизировать обработку ошибок

3.2 Улучшение пользовательского опыта

- Добавить индикаторы загрузки
- Улучшить сообщения об ошибках
- Оптимизировать переключение между режимами

Этап 4: Финальное тестирование (20 минут)

4.1 Тестирование всех режимов

- **Локальный режим:** Qwen2-VL-2B через Python сервер
- **Облачный режим:** OpenRouter API с различными моделями
- **Гибридный режим:** Автоматическое переключение

4.2 Тестирование Touch Grid системы

- Проверить точность координат
- Протестировать на разных размерах экрана
- Валидировать интеграцию с браузером

4.3 Тестирование веб-автоматизации

- Проверить Python компоненты
- Протестировать Control Room интерфейс
- Валидировать OCR функциональность

🛠️ Детальная реализация

Создание Python-iOS моста

Файл: `qwen2vl_server.py`

```python
from flask import Flask, request, jsonify
from mlx_vlm import load, generate
import base64
from PIL import Image
import io

app = Flask(__name__)
model, processor = None, None

@app.route('/analyze', methods=['POST'])
def analyze_image():
    data = request.json
    image_data = base64.b64decode(data['image'])
    prompt = data['prompt']

    image = Image.open(io.BytesIO(image_data))
    response = generate(model, processor, image, prompt)

    return jsonify({'response': response})

if __name__ == '__main__':
    model, processor = load("mlx-community/Qwen2-VL-2B-4bit")
    app.run(host='localhost', port=8888)
```

Обновление `Qwen2VLManager.swift`

```swift
private func performQwen2VLInference(image: UIImage, prompt: String, task: UIAutoma-
tionTask) async -> UIAutomationResult {
    guard let imageData = image.jpegData(compressionQuality: 0.8) else {
        return UIAutomationResult(success: false, error: "Failed to convert image")
    }

    let base64Image = imageData.base64EncodedString()
    let requestBody = [
        "image": base64Image,
        "prompt": createVisionLanguagePrompt(for: task, with: prompt)
    ]

    // HTTP запрос к Python серверу
    let url = URL(string: "http://localhost:8888/analyze")!
    var request = URLRequest(url: url)
    request.httpMethod = "POST"
    request.setValue("application/json", forHTTPHeaderField: "Content-Type")
    request.httpBody = try? JSONSerialization.data(withJSONObject: requestBody)

    let (data, _) = try await URLSession.shared.data(for: request)
    let response = try JSONSerialization.jsonObject(with: data) as! [String: Any]

    return parseRealModelResponse(response["response"] as! String, for: task, in: im-
age)
}
```

Улучшение пользовательского интерфейса

Добавление индикатора состояния модели

```
struct ModelStatusView: View {
    @ObservedObject var qwen2VLManager = Qwen2VLManager.shared

    var body: some View {
        HStack {
            Circle()
                .fill(qwen2VLManager.isModelLoaded ? .green : .red)
                .frame(width: 8, height: 8)

            Text(qwen2VLManager.isModelLoaded ? "Qwen2-VL Ready" : "Loading...")
                .font(.caption)
        }
    }
}
```

📋 Чек-лист выполнения

Подготовка

- [ ] Установить MLX и зависимости
- [ ] Проверить доступность модели Qwen2-VL-2B
- [ ] Создать необходимые директории

Интеграция модели

- [ ] Загрузить и протестировать MLX модель
- [ ] Создать Python HTTP сервер
- [ ] Обновить Swift код для реальных запросов
- [ ] Протестировать интеграцию

Оптимизация OpenRouter

- [ ] Добавить инструкции по API ключам
- [ ] Протестировать все модели
- [ ] Улучшить обработку ошибок
- [ ] Оптимизировать UI

Финальное тестирование

- [ ] Протестировать все режимы работы
- [ ] Проверить Touch Grid систему
- [ ] Валидировать веб-автоматизацию
- [ ] Провести интеграционные тесты

Документация

- [ ] Обновить README с инструкциями
- [ ] Создать руководство по настройке
- [ ] Добавить примеры использования
- [ ] Обновить архитектурную документацию

⚡ Быстрый старт (для опытных разработчиков)

```
# 1. Установка зависимостей
pip install mlx-vlm flask pillow

# 2. Запуск Python сервера
python qwen2vl_server.py

# 3. Сборка iOS приложения
open EirosAgent.xcodeproj
# Build and Run (Cmd+R)

# 4. Тестирование
# - Переключиться в локальный режим
# - Сделать скриншот
# - Проверить анализ через Qwen2-VL
```

🎯 Ожидаемые результаты

После выполнения плана:

✅ Функциональность

- **100% рабочая интеграция** с реальной моделью Qwen2-VL-2B
- **Три режима работы:** Локальный, Облачный, Гибридный
- **Точная UI автоматизация** с координатами элементов
- **Полная веб-автоматизация** через Python framework

✅ Производительность

- **Время инференса:** 0.5-1.5 секунды на анализ экрана
- **Точность:** 90%+ для UI элементов
- **Память:** <400MB для локальной модели
- **Стабильность:** Graceful fallback при ошибках

✅ Пользовательский опыт

- **Интуитивный интерфейс** с индикаторами состояния
- **Быстрое переключение** между режимами
- **Подробная обратная связь** о процессе анализа
- **Надежная работа** с обработкой ошибок

🚨 Важные замечания

Системные требования

- **iOS 16+** для приложения
- **Python 3.8+** для MLX сервера
- **8GB RAM** рекомендуется для локальной модели
- **Интернет** для OpenRouter API

Безопасность

- **Локальная обработка** для приватных данных

- **Secure Keychain** для API ключей
- **HTTPS** для всех внешних запросов
- **Валидация входных данных**

Производительность

- **Lazy loading** моделей
- **Кэширование** результатов
- **Batch processing** для множественных запросов
- **Memory management** для стабильности

🎉 **Готово к выполнению!** Этот план восстановит полную функциональность проекта и выведет его на продакшн уровень.

Для доступа к приватным репозиториям: GitHub App Configuration (https://github.com/apps/abacusai/installations/select_target)